

Mini-Blog – Backend Spring Boot

Rapport d'Architecture, Conception et Choix Techniques

Équipe de Développement & Conception

25 septembre 2026

Résumé

Ce document présente l'architecture logicielle, le modèle relationnel et l'ensemble des choix de conception retenus pour le développement du backend du projet **Mini-Blog**. Conçu avec **Spring Boot 3**, **Java 21**, **PostgreSQL 16** et conteneurisé via **Docker**, ce backend a été pensé pour servir d'API REST robuste à un futur client **Angular**. Une attention particulière a été portée à la sécurité granulaire au niveau des ressources (gestion des droits selon l'auteur et le statut de publication DRAFT/PUBLISHED) ainsi qu'à la reproductibilité du déploiement. Ce rapport formalise et justifie l'ensemble des décisions structurantes issues de la phase d'alignement technique.

Table des matières

1	Introduction et Périmètre du Projet	2
2	Choix d'Architecture et Justifications Techniques	2
2.1	Axe 1 : Mécanisme d'Authentification et Gestion de Session	2
2.2	Axe 2 : Inscription et Amorçage du Rôle Administrateur	2
2.3	Axe 3 : Règles de Gestion et Modération des Commentaires	3
2.4	Axe 4 : Cycle de Vie et Édition des Articles (Politique Stricte)	3
2.5	Axe 5 : Endpoints de Workflow pour la Publication	3
2.6	Axe 6 : Filtrage et Consultation des Articles	4
2.7	Axe 7 : Stratégie de Suppression des Données	4
2.8	Axe 8 : Gestion du Schéma de Base de Données	4
2.9	Axe 9 : Conteneurisation et Outillage de Documentation	5
3	Modèle de Données et Schéma Relationnel	5
4	Matrice de Sécurité et Contrôle d'Accès	5
5	Architecture Logicielle Spring Boot	5
6	Conclusion et Prochaines Étapes	7

1 Introduction et Périmètre du Projet

Le projet **Mini-Blog** a pour objectif d'offrir une plateforme moderne de publication et d'interaction autour d'articles de blog. L'application est architecturée selon un découpage découplé en deux tiers :

- Un backend sous forme d'API REST sécurisée développée en Java avec le framework **Spring Boot 3**.
- Une base de données relationnelle **PostgreSQL** pour garantir l'intégrité et la persistance des données.
- Un futur frontend en **Angular** qui consommera cette API.

Ce premier jalon concerne intégralement la réalisation du backend, de son infrastructure de conteneurisation Docker, de ses tests automatisés et de sa documentation OpenAPI/Swagger.

—

2 Choix d'Architecture et Justifications Techniques

Lors de la phase de cadrage et d'interview technique, plusieurs axes structurants ont été arbitrés. Cette section détaille pour chaque axe le contexte, les options étudiées et la justification du choix final.

2.1 Axe 1 : Mécanisme d'Authentification et Gestion de Session

Contexte : L'API backend est vouée à être consommée par une application Single Page (Angular) et doit pouvoir supporter une montée en charge sans adhérence à une session serveur.

Options étudiées :

1. **JWT stateless (Bearer Token dans l'en-tête Authorization)** (Retenu).
2. JWT transmis via cookies `HttpOnly` / `SameSite`.
3. Session HTTP standard avec cookie d'état `JSESSIONID`.

Justification du choix : Le token JWT stateless transmis par en-tête HTTP est le standard de l'industrie pour les architectures SPA / API REST. Il supprime tout état sur le serveur (stateless), facilite les déploiements horizontaux sans session distribuée, évite les problématiques complexes de CSRF inhérentes aux cookies et simplifie l'outillage de test (Swagger UI, Postman, curl).

2.2 Axe 2 : Inscription et Amorçage du Rôle Administrateur

Contexte : Le système distingue deux rôles utilisateurs : `ROLE_USER` et `ROLE_ADMIN`. Il est nécessaire de définir la procédure d'inscription publique et la création du premier compte administrateur.

Options étudiées :

1. **Inscription publique pour USER + Seeding automatique d'un ADMIN par défaut au démarrage** (Retenu).
2. Inscription publique pour `USER` + promotion manuelle via SQL.
3. Inscription fermée (création manuelle par un administrateur préexistant).

Justification du choix : L'inscription ouverte via `POST /api/auth/register` permet l'autonomie des utilisateurs réguliers. Pour l'administration, l'implémentation d'un composant `Spring DataInitializer` contrôlé par les propriétés d'environnement (`application.yml` / `Docker env`) assure que l'administrateur initial est créé automatiquement dès le premier démarrage s'il n'existe pas déjà, garantissant un environnement prêt à l'emploi.

2.3 Axe 3 : Règles de Gestion et Modération des Commentaires

Contexte : Les commentaires constituent la couche d'interaction du blog. Le système doit définir qui peut commenter et qui peut modifier ou supprimer un commentaire.

Options étudiées :

1. **Commentaires réservés aux utilisateurs connectés sur articles PUBLISHED, modification/suppression par l'auteur, suppression par l'admin** (Retenu).
2. Commentaires sans modification possible (immuables) avec suppression par l'auteur ou l'admin.
3. Commentaires ouverts aux visiteurs anonymes avec modération a priori.

Justification du choix : Interdire les commentaires aux anonymes prévient le spam et les abus. Restreindre les commentaires aux seuls articles ayant le statut `PUBLISHED` préserve la cohérence : un article en cours de rédaction (`DRAFT`) ne doit pas recevoir de commentaires publics. La possibilité pour l'auteur de corriger ou retirer son commentaire, combinée au pouvoir de modération absolue de l'administrateur, offre un équilibre idéal.

2.4 Axe 4 : Cycle de Vie et Édition des Articles (Politique Stricte)

Contexte : Un article possède deux statuts : `DRAFT` et `PUBLISHED`. La question centrale réside dans les droits de l'auteur une fois que son article a été validé et publié par un administrateur.

Options étudiées :

1. **Politique Stricte : l'auteur ne peut modifier/supprimer son article que s'il est en DRAFT. Une fois PUBLISHED, seul l'admin peut modifier, supprimer ou dépublier** (Retenu).
2. Revalidation automatique : toute modification par l'auteur d'un article publié le repasse automatiquement en `DRAFT`.
3. Édition libre par l'auteur sur l'article publié sans repasser par l'admin.

Justification du choix : La politique stricte garantit l'intégrité éditoriale de la plateforme. Un administrateur ayant validé une version précise d'un article, un utilisateur ne peut pas altérer le contenu sans contrôle éditorial. Si un article doit être modifié, l'administrateur conserve la possibilité de le modifier directement ou de le repasser en `DRAFT`.

2.5 Axe 5 : Endpoints de Workflow pour la Publication

Contexte : Comment matérialiser l'action de validation/publication dans l'API REST.

Options étudiées :

1. **Endpoints dédiés : PATCH /api/articles/{id}/publish et /unpublish** (Retenu).
2. Envoi d'un champ `{"status": "PUBLISHED"}` sur le `PATCH /api/articles/{id}` classique.
3. Workflow étendu avec motif de rejet.

Justification du choix : Définir des endpoints dédiés apporte une sémantique REST explicite et claire. Cela permet d'appliquer une sécurité déclarative stricte (`@PreAuthorize("hasRole('ADMIN')")`) sur ces actions spécifiques sans risquer qu'un utilisateur n'injecte un changement de statut lors d'une simple mise à jour de texte.

2.6 Axe 6 : Filtrage et Consultation des Articles

Contexte : Les articles en DRAFT ne doivent être visibles que par leur auteur et par les administrateurs, tandis que les articles PUBLISHED sont publics.

Options étudiées :

1. **Endpoint unique GET /api/articles avec filtrage sécurisé selon le profil connecté et pagination Spring Data** (Retenu).
2. Endpoints scindés (`/api/articles`, `/api/articles/me`, `/api/admin/articles`).
3. Endpoint public rejetant tout brouillon sauf avec paramètre explicite.

Justification du choix : Un endpoint unifié avec pagination et filtres optionnels (`?status=...`) centralise la logique de recherche. La couche de service et le repository appliquent automatiquement les prédicats de visibilité :

- Visiteur anonyme : `status = 'PUBLISHED'`.
- Utilisateur authentifié : `status = 'PUBLISHED' OR (status = 'DRAFT' AND author = currentUser)`.
- Administrateur : accès à tous les articles, avec possibilité de filtrer par statut (par exemple pour afficher la file d'attente des brouillons à valider).

2.7 Axe 7 : Stratégie de Suppression des Données

Contexte : Déterminer le comportement du système lors de la suppression d'un article ou d'un commentaire.

Options étudiées :

1. **Suppression physique (Hard delete) avec cascade SQL sur les commentaires** (Retenu).
2. Suppression logique (Soft delete) avec colonne `deleted_at`.
3. Approche mixte (hard delete sur les brouillons, soft delete sur les articles publiés).

Justification du choix : Le modèle relationnel avec contrainte `ON DELETE CASCADE` sur la table `comments` garantit la propreté de la base de données sans laisser d'enregistrements orphelins. La suppression physique est simple, déterministe et conforme aux exigences de droit à l'oubli.

2.8 Axe 8 : Gestion du Schéma de Base de Données

Contexte : Assurer la création et l'évolution du schéma de base de données PostgreSQL de manière reproductible entre les environnements de développement, de test et de conteneurisation.

Options étudiées :

1. **Migrations SQL versionnées avec Flyway** (Retenu).
2. Génération automatique Hibernate DDL (`ddl-auto: update`).
3. Changelogs Liquibase.

Justification du choix : Flyway applique des scripts SQL natifs (`V1__init_schema.sql`) de façon prévisible et audité. Contrairement à `ddl-auto: update` d'Hibernate (fortement déconseillé en environnement réel), Flyway assure un contrôle strict des types, index, contraintes de clés étrangères et cascades.

2.9 Axe 9 : Conteneurisation et Outillage de Documentation

Contexte : Faciliter l'exécution du backend pour les développeurs et préparer l'intégration de la future application Angular.

Options étudiées :

1. **Docker Compose complet (PostgreSQL + Spring Boot) & Swagger UI (Springdoc OpenAPI)** (Retenu).
2. Docker Compose restreint à PostgreSQL avec exécution locale de Spring Boot.
3. Docker Compose sans interface Swagger UI.

Justification du choix : Le fichier `docker-compose.yml` permet d'initialiser tout le système en une seule commande (`docker compose up`). L'intégration de Swagger UI (`/swagger-ui/index.html`) avec configuration du schéma de sécurité `BearerAuth` permet d'interagir immédiatement avec l'API, de s'authentifier et d'éprouver la matrice des droits avant même le développement de l'interface Angular.

3 Modèle de Données et Schéma Relationnel

Le schéma de la base de données est composé de trois tables principales :

4 Matrice de Sécurité et Contrôle d'Accès

La matrice ci-dessous résume les permissions implémentées dans le backend :

TABLE 1 – Matrice des Droits d'Accès de l'API

Opération / Ressource	Anonyme	Utilisateur (USER)	Administrateur (ADMIN)
Inscription (<code>/api/auth/register</code>)	✓	✓	✓
Connexion (<code>/api/auth/login</code>)	✓	✓	✓
Voir articles PUBLISHED	✓	✓	✓
Voir articles DRAFT	×	Auteur uniquement	Tous les auteurs
Créer un article	×	✓ (forcé en DRAFT)	✓ (forcé en DRAFT)
Modifier article en DRAFT	×	Auteur uniquement	✓
Supprimer article en DRAFT	×	Auteur uniquement	✓
Modifier article PUBLISHED	×	×	✓
Supprimer article PUBLISHED	×	×	✓
Publier (DRAFT → PUBLISHED)	×	×	✓
Dépublier (PUBLISHED → DRAFT)	×	×	✓
Lire les commentaires d'un article	✓	✓	✓
Commenter un article PUBLISHED	×	✓	✓
Commenter un article DRAFT	×	×	×
Modifier un commentaire	×	Auteur uniquement	Auteur uniquement
Supprimer un commentaire	×	Auteur uniquement	✓ (Modération totale)

Structure des Tables SQL (Flyway)

Table users

- id : BIGSERIAL PRIMARY KEY
- email : VARCHAR(150) NOT NULL UNIQUE
- password : VARCHAR(255) NOT NULL (hash BCrypt)
- role : VARCHAR(20) NOT NULL (ROLE_USER, ROLE_ADMIN)
- created_at : TIMESTAMP WITH TIME ZONE NOT NULL

Table articles

- id : BIGSERIAL PRIMARY KEY
- title : VARCHAR(255) NOT NULL
- content : TEXT NOT NULL
- status : VARCHAR(20) NOT NULL (DRAFT, PUBLISHED)
- author_id : BIGINT NOT NULL REFERENCES users(id)
- created_at : TIMESTAMP WITH TIME ZONE NOT NULL
- updated_at : TIMESTAMP WITH TIME ZONE NOT NULL

Table comments

- id : BIGSERIAL PRIMARY KEY
- content : TEXT NOT NULL
- article_id : BIGINT NOT NULL REFERENCES articles(id) ON DELETE CASCADE
- author_id : BIGINT NOT NULL REFERENCES users(id)
- created_at : TIMESTAMP WITH TIME ZONE NOT NULL
- updated_at : TIMESTAMP WITH TIME ZONE NOT NULL

5 Architecture Logicielle Spring Boot

L'application respecte les principes de l'architecture en couches (*Layered Architecture*) :

1. **Couche Présentation (Controllers)** : Définition des endpoints REST, validation des requêtes DTOs via Bean Validation (@Valid, @NotBlank, @Email), sérialisation JSON et documentation OpenAPI.
2. **Couche Sécurité & Filtres** : JwtAuthenticationFilter interceptant les requêtes, déchiffrant le JWT via JwtService et alimentant le SecurityContextHolder. Les évaluateurs d'autorisation personnalisés (ArticleSecurity, CommentSecurity) appliquent les règles de propriété des ressources.
3. **Couche Métier (Services)** : Implémentation des règles fonctionnelles, vérification de l'intégrité (statut DRAFT vs PUBLISHED) et gestion transactionnelle (@Transactional).
4. **Couche Persistance (Repositories)** : Interfaces Spring Data JPA exploitant des requêtes JPQL sécurisées pour le filtrage contextuel et la pagination.
5. **Couche Infrastructure (Flyway / Docker)** : Initialisation du schéma SQL et conteneurisation reproductible.

6 Conclusion et Prochaines Étapes

Ce backend fournit un socle fiable, sécurisé et totalement testé. L'exhaustivité de la matrice des droits et la clarté des contrats d'API exposés via Swagger UI permettront d'aborder la phase

suivante (le développement du client Angular) avec un niveau élevé de prédictibilité et de confort d'intégration. Ce rapport sera enrichi au fil des itérations futures du projet.